Virtual Pointer – projektötlet / megvalósítási terv

A feladat lényege:

A cél egy olyan rendszer készítése, amelyben az előadó mutatóujjának iránya alapján meghatározza, hogy a kivetítő/tábla melyik pontjára mutat, majd oda egy virtuális pointert rajzol.
A pointer ne csak a saját programunkban jelenjen meg, hanem egy Windows-os overlayként, ezzel mindenhol látható legyen.

A fő ötlet:

2 kamera → kéz/ujj felismerése → 3D irány meghatározása → tábla síkjával való metszés → pointer pozíció → Windows overlay

Elhelyezés:

-Nagyjából tölcsért formálva VAGY az egyik nagyobb szögben, oldalról.

Szoftverstack

Python, OpenCV, NumPy, MediaPipe (https://mediapipe.readthedocs.io/en/latest/solutions/hands.html)
PySide6 / Qt
https://doc.qt.io/qtforpython-6/
https://doc.qt.io/qtforpython-6.8/PySide6/QtWidgets/QWidget.html
Kalman filter

Kapcsolódó Kató Zoltán anyagok

1. Camera Geometry
https://www.inf.u-szeged.hu/~kato/teaching/computervision/02-CameraGeometry.pdf
A pinhole kamera, perspektív projekció, fókusztávolság, látómező és 3D → 2D projekció alapjai.
→ Ez kell ahhoz, hogy értsük, hogyan keletkezik a kamera képéből a térbeli információ.
2. Camera Calibration
https://www.inf.u-szeged.hu/~kato/teaching/computervision/03-CameraCalibration.pdf
Belső/külső kamera-paraméterek, projekciós mátrix, pont megfeleltetések, DLT és SVD.
→ Ezzel kalibráljuk a két kamerát.
3. Stereo
https://www.inf.u-szeged.hu/~kato/teaching/computervision/05-Stereo.pdf
Két kamerából történő 3D rekonstrukció, megfeleltetés és trianguláció.
→ Ez adja a két kamerás rendszer geometriai alapját.
4. Tracking
https://www.inf.u-szeged.hu/~kato/teaching/computervision/09-Tracking.pdf
Képpontok/objektumok követése videóban, KLT, Kalman filter stb.
→ Ezt használjuk a pointer stabilizálásához és adott esetben a kéz követéséhez.
Teljes anyag: https://www.inf.u-szeged.hu/~kato/teaching/computervision/SzamitogepesLatas.pdf

A kamerák kalibrációja, az R és T paraméterek meghatározása

Cél: A két kamera belső optikai paramétereinek, lencsetorzításának és egymáshoz viszonyított térbeli helyzetének
meghatározása. Bemenet: Szinkronizált képpárok egy mozgatott sakktábláról ($X_i \leftrightarrow x_i$). Kimenet: K_1, K_2 belső mátrixok, D_1, D_2 torzítási vektorok, R, T relatív transzformáció, valamint a P_1, P_2 projekciós mátrixok.

1. lépés – Kameramodell és DLT / Gold Standard kalibráció: Egy térbeli $X_i=[X_i,Y_i,Z_i,1]^T$ világpontnak a képsíkra történő $x_i=[x_i,y_i,1]^T$ vetületét a 3x4-es P=K[R|t] projekciós mátrix írja le, ahol K a fókuszpontokat (f_x,f_y), a főpontokat (p_x,p_y) és a tengelydőlést (s) tartalmazó belső mátrix, R a 3x3-as ortogonális forgatási mátrix, t pedig az eltolási vektor. Az $x_i \times PX_i=0$ => Ap=0 összefüggésből minden pontpár 2 független egyenletet ad. Megoldás (n>=pontpárra): Kezdeti P becslése ||Ap|| minimalizálásával (SVD / DLT), majd a

minpSumi $d(x_i,PX_i)^2$ visszavetítési hiba és a D_1,D_2 lencsetorzítások finomítása *Levenberg–Marquardt* iterációval.

2. lépés – Sztereó R,T transzformáció: Rögzítjük a közös 3D világkoordináta rendszer origóját az első kamerába ($R=I,t_1=0$), a 2. kamera helyzetét az 1. kamerához viszonyított relatív R forgatás és T eltolás adja meg: $P_1=K_1[I|0]$, $P_2=[R|T]$ ($x=K^{-1}x$ norm. koordináták közötti epipoláris kényszert az $E=[T]xR$ esszenciális mátrix írja le: $\underline{x}'^{T}E\underline{x}=0$)

Függvényterv: $[K_1,D_1,K_2,D_2,R,T,imgSize]$=*stereoParametersToOpenCS(stereoParams)* (MATLAB) és *load_stereo_calibration(config_path)* (Python). A MATLAB kalibráció eredményét exportáljuk, a Python függvény beolvassa a paramétereket és felépíti a P_1, P_2 mátrixokat. Folyamat röviden: szinkronizált képek → MATLAB sztereó kalibráció → OpenCV export → projekciós mátrixok számítása

2. A tábla térbeli meghatározása

Cél: A tábla 3D síkjának és lokális koordináta-rendszerének meghatározása az 1. kamera terében külső pózbecsléssel. Bemenet: A tábla fizikai mérete (L_w,L_h), 4 sarokpont képi koordinátái (cam1) és K_1,D_1. Kimenet: Porigin (táblaorigó), uaxis, vaxis (egységvektorok), (a,b,c,d) síkegyütthatók.

Számítási lépések:

1. Lokális 3D sarokpotok (CWM): Tábla síkja Z=0, a négy sarokpont: $B_1=(0,0,0)$, $B_2=(L_W,0,0)$, $B_3=(L_w,L_h,0)$, $B_4=(0,L_h,0)$
2. Külső orientáció becslése: Sarokpontok kijelölése az 1. kamera torzításmentesített (UDP) képén (manuálisan/ArUco-markerek): $x_i=(x_i,y_i)$, $i\in\{1,\ldots,4\}$ (CPM). Az estimateExtrinsics(CPM,CWM,UDP) meghatározza a tábla rendszeréből a kamera retébe mutató (R_{board},T_{board}) transzformációt: $B_{i,cam}=R_{board}B_i^{T}+T_{board}$, $i\in\{1,\ldots,4\}$
3. Sík és tengelyek felírása: A kamera-térbe transzformált $B_{1,cam}$, $B_{2,cam}$, $B_{4,cam}$ pontokból a metszéspont-számításhoz szükséges paraméterek: Tábla origó: Porigin = $B_{1,cam}$; Helyi tengelyek e.v.-i: $u_{axis}=(B_{2,cam}-B_{1,cam})/(||B_{2,cam}-B_{1,cam}||)$, $v_{axis}=(B_{4,cam}-B_{1,cam})/(||B_{4,cam}-B_{1,cam}||)$; Sík normálvektora és eltolása: $(ax+by+cz+d=0):n=(a,b,c)T=u_{axis}*v_{axis}$, $d=-(n*P_{origin})$

Függvényterv: *compute_board_geometry(corners_2d, L_w, L_h, K_1, D_1)* – torzításmentesíti a 4 sarokpontot, pózbecsléssel

(*estimateExtrinsics*) meghatározza R_{board}, T_{board} értékét, majd kiszámítja és visszaadja Porigin,uaxis,vaxis és plane_coeffs=(a,b,c,d)

paramétereket. Folyamat röviden: 4 sarok és (L_w, L_h) → [pózbecslés] → Bi,cam → Porigin,uaxis,vaxis,(a,b,c,d)

3. A mutatóujj 2D pozíciójának meghatározása

Cél: A triangulációhoz szükséges megfeleltetési probléma megoldása: ujjpontok vetületeinek valós idejű detektálása mindkét kamera képén. Bemenet: Két kamera aktuális képkockája, K_1,D_1,K_2,D_2. Kimenet: Torzításmentesített 2D landmark-párok.

Számítási lépések:

1. Kulcspontok detektálása: MediaPipe-al mindkét képen megkeressük a kezet és kinyerjük a mutatóujj pontjait: INDEX_FINGER_MCP, PIP, DIP, TIP
2. Pixelkoordinátákra váltás és torzításkorrekció: normalizált koordináták felszorzása a képfelbontással → homogén pixelkoordináták: xi=[xi,yi,1]T, x'i=[x'i,y'i,1]T, i {MCP,PIP,DIP,TIP}; trianguláció lineáris kameramodell t ∈feltételez (x=PX) → cv2.undistortPoints() lencsetorzítási korrekció

Függvényterv: detect_finger_landmarks(frame_cam1, frame_cam2, K_1, D_1, K_2, D_2) – két képkockán lefuttatja a *Hands* modellt, kinyeri a mutatóujj (x,y) pixelkoordinátáit a cv2.undistortPoints() hívással korrigálja a lencsetorzítást, és visszaadja a landmarks_cam1, landmarks_cam2 szótárakat a triangulate_landmarks() számára.

Folyamat röviden: Képkockák (cam1, cam2)→MediaPipe Hands→[pixelre skálázás] → cv2.undistortPoints()→2D landmark-párok (x'_i,y'_i)

Mutatás-detektálás: 2D landmarkokból 3D mutatási sugár

Cél: két kamera képén detektált kéz-landmarkokból (MediaPipe: INDEX_FINGER_DIP, INDEX_FINGER_TIP) kiszámolni, merre mutat az előadó. Az eredmény egy 3D sugár, amelyet a következő lépés a tábla síkjával metsz. **Bemenet:** 2D landmarkok mindkét kamerából (ugyanaz a képkocka), P_1, P_2 ($P = P_1$, $P' = P_2$) projekciós mátrixok.

Kimenet: sugár kezdőpontja (P_{TIP}) és egységnyi iránya ($\hat{v}$).

1. lépés – 2D landmarkok triangulálása 3D pontokká

A két kamera más szögből látja a kezet, így ugyanaz a landmark két különböző képpontként jelenik meg. A kalibrációs paraméterekből és a megfelelő képpontpárból kiszámítható a pont 3D helyzete a közös koordinátarendszerben (lineáris triangulálás, SVD). Ezt minden szükséges landmarkra (min. DIP és TIP) külön elvégezzük.

Kameramodell: $x = P \cdot X$, $x' = P' \cdot X$ (homogén koordináták; $x=[x\ y\ 1]^T$ a 2D képpont, $X=[X\ Y\ Z\ W]^T$ a 3D pont). $P = K \cdot [R \mid t]$, $t = -R \cdot \tilde{C}$. ahol a **K**: belső mátrix (fx,fy fókusz; px,py főpont; s tengelydőlés, digitális kam. ≈ 0), **R**: forgatás, **t**: eltolás, $\tilde{C}$: kamera középpontja a világkoordinátában. Ha az 1. kamera az origó: $R_1=I$, $t_1=0$; a 2. kamerára $R_2=R$, $t_2=T$ (relatív pózból).

Egyenletek: P sorai p^{1T}, p^{2T}, p^{3T} (P'-é p'^{1T}, p'^{2T}, p'^{3T}). Az $x \times PX = 0$ összefüggésből kameránként 2 független lineáris egyenlet jön:

$x \cdot (p^{3T}X) - p^{1T}X = 0$	$y \cdot (p^{3T}X) - p^{2T}X = 0$	(1. kamera, (x,y))
$x' \cdot (p'^{3T}X) - p'^{1T}X = 0$	$y' \cdot (p'^{3T}X) - p'^{2T}X = 0$	(2. kamera, (x′,y′))

A 4 egyenlet egy homogén rendszer: **$A \cdot X = 0$**, ahol A (4×4) sorai:

$$A = [\ x \cdot p^{3T} - p^{1T}\ ;\ y \cdot p^{3T} - p^{2T}\ ;\ x' \cdot p'^{3T} - p'^{1T}\ ;\ y' \cdot p'^{3T} - p'^{2T}\ ]$$

Megoldás: SVD-vel. A legkisebb szinguláris értékhez tartozó jobb szinguláris vektor adja $X_h=[X_h\ Y_h\ Z_h\ W_h]^T$-t; a valódi koordináták: $\mathbf{X = X_h/W_h,\ Y = Y_h/W_h,\ Z = Z_h/W_h}$. OpenCV-ben ezt a **cv2.triangulatePoints()** végzi el.

Előfeltételek: (a) a képpontok illeszkedjenek a kalibrációhoz; torzított képnél előbb torzításkorrekció, ha a **cv2.undistortPoints()** normalizált koordinátákat ad, a triangulációhoz belső paraméterektől mentes P-t ($P=[R|t]$, $K=I$) kell használni; (b) a két pont ugyanahhoz a képkockához tartozzon; (c) valóban ugyanazt a landmarkot jelöljék.

Függvényterv: *triangulate_landmarks(landmarks_cam1, landmarks_cam2, P1, P2)*

A függvény bemenetként megkapja a két kamera megfelelő 2D landmarkjait (pixelkoordináták, (x, y)) és a kamerák P_1, P_2 projekciós mátrixait. Minden landmarkpárra meghívja a `cv2.triangulatePoints()` függvényt, a kapott homogén koordinátákat elosztja a negyedik komponenssel ($X = X_h/W_h$, $Y = Y_h/W_h$, $Z = Z_h/W_h$), és visszaadja a 3D pontokat. A visszatérési érték egy map, amelyben a landmark neve (INDEX_FINGER_DIP, INDEX_FINGER_TIP stb.) a hozzá tartozó (X, Y, Z) koordinátához tartozik.

2. lépés – A mutatás irányvektorának meghatározása

A triangulált 3D pontokból az ujj irányát számoljuk: a DIP (ujjhegy előtti ízület) és a TIP (ujjhegy) pontot használjuk (szükség esetén a csukló vagy a közbenső pontok is bevonhatók a pontosság javítására). Az irányvektor a következő lépésben a tábla síkjának metszéséhez kell.

1) Pontok: $P_{DIP}=[X_D\ Y_D\ Z_D]^T$, $P_{TIP}=[X_T\ Y_T\ Z_T]^T$ (a triangulált 3D pontok). **2) Irány:** $v = P_{TIP} - P_{DIP}$. **3) Normalizálás:** $\hat{v} = v/\|v\|$ (hossza 1, iránya azonos). Előtte ellenőrizni: $\|v\|$ ne legyen nulla vagy ahhoz közeli (különben a landmark-zaj miatt értelmetlen irány jön).

4) Mutatási sugár: $\mathbf{r(\lambda) = P_{TIP} + \lambda \cdot \hat{v},\ \lambda \geq 0}$. Ahol $\mathbf{P_{TIP}}$ a kezdőpont, $\hat{\mathbf{v}}$ az irány, $\boldsymbol{\lambda}$ a sugár mentén megtett távolság, a $\lambda \geq 0$ feltétel biztosítja, hogy a találat az ujj előtt van, nem mögötte.

Függvényterv: *calculate_pointing_direction(points_3d)*

A függvény a triangulációból kapott 3D landmarkokat használja, különösen az INDEX_FINGER_DIP és INDEX_FINGER_TIP pontokat. Kiszámítja a két pont különbségéből a $v = P_{TIP} - P_{DIP}$ irányvektort, ellenőrzi, hogy a hossza nem nulla vagy közel nulla, majd egységnyi hosszúra normalizálja ($\hat{v} = v/\|v\|$). Ha a vektor hossza túl kicsi, nem ad vissza érvényes irányt. Visszatérési értéke az P_{TIP} kezdőpont és a normalizált $\hat{v}$ irányvektor, amelyet a következő függvény a tábla síkjával való metszéspont meghatározásához használhat.

Folyamat röviden

2D landmarkok (cam1, cam2) → [torzításkorrekció] → triangulate_landmarks() → 3D pontok → calculate_pointing_direction() → sugár (P_{TIP}, $\hat{v}$) → (következő lépés) metszés a tábla síkjával.

Mutatóujj és a tábla síkjának metszése (3D pont meghatározása)

Cél: a mutatóujj által meghatározott 3D sugár (**P_{TIP}** és **$\hat{v}$**) és a tábla síkjának (**ax+by+cz+d=0**) metszéspontjának kiszámítása a 3D térben.

Paraméterek: az ujj kezdőpontja **$P_{TIP} = [x_0, y_0, z_0]$**, az ujj irányvektora **$\hat{v} = [v_x, v_y, v_z]$**

a sík normálvektora **(a, b, c)** és az eltolás mértéke **d** az első kamera origójához képest

Sugár képlete: $r(\lambda) = P_{TIP} + \lambda \cdot \hat{v}$, $\lambda \geq 0$ Sík képlete: $ax + by + cz + d = 0$

Metszéspont Elméleti Számítása:

1. sugár koordinátáit kiszámoljuk: $x = x_0 + \lambda v_x$, $y = y_0 + \lambda v_y$, $z = z_0 + \lambda v_z$
2. behelyettesítünk a sík egyenletébe: $a(x_0 + \lambda v_x) + b(y_0 + \lambda v_y) + c(z_0 + \lambda v_z) + d = 0$
3. λ paraméter kiszámítása: $\lambda = (ax_0 + by_0 + cz_0 + d) / (av_x + bv_y + cv_z)$
4. 3D metszéspont meghatározása: $P_{INTERSECT} = P_{TIP} + \lambda \hat{v}$

Előfeltételek és Szélsőséges esetek:

Ha a **λ** számításánál a nevező **$(av_x + bv_y + cv_z)$** közel **0**, a sugár közel párhuzamos a síkkal így nincsen metszéspont.

Ha **$\lambda < 0$** a metszéspont az ujj mögött van, így érvénytelen a mutatási irány szempontjából

Függvényterv: *calculate_intersection(p_tip, v_hat, plane_coeffs)*

Felbontja a paramétereket: **a, b, c, d = plane_coeffs; x0, y0, z0 = p_tip; vx, vy, vz = v_hat**

Kiszámolja a nevezőt **divider = a * vx + b * vy + c * vz** és ha közel 0, **None**-t ad vissza

Kiszámolja **lambda = (a * x0 + b * y0 + c*z0 + d) / divider** ha negatív **None**-t ad vissza

Kiszámolja és visszaadja **p_intersect = p_tip + lambda * v_hat**

Folyamat röviden

sugár (P_{TIP}, $\hat{v}$) és (a,b,c,d sík együtthatók) → [távolság számítás] → [3D pont] → $P_{INTERSECT}$

A 3D metszéspont transzformálása a tábla 2D koordinátarendszerébe és pixelkoordinátára képzése

Cél: a tábla síkján kapott 3D metszéspont ($P_{INTERSECT}$) átalakítása a kijelző képpontjára

Paraméterek: P_{ORIGIN} (tábla origója), **u_{axis},v_{axis}** (tábla 2D egységvektorai), **L_w, L_h** (tábla fizikai szélessége és magassága), valamint **SCR_WIDTH**, **SCR_HEIGHT** (képernyő felbontása)

Számítási Lépések:

1. eltolási vektor kiszámítása: $w = P_{INTERSECT} - P_{ORIGIN}$
2. vetítés a helyi tengelyekre (skaláris szorzat): $x_{2D} = w \cdot u_{axis}$, $y_{2D} = w \cdot v_{axis}$

3. normalizálás (arányok kiszámítása 0 és 1 között): $\mathbf{x_{norm} = x_{2D} / L_w}$, $\mathbf{y_{norm} = y_{2D} / L_h}$
4. értékek korlátozása (clamping) a [0, 1] tartományra → túlindexelés elkerülése
5. pixelkoordinátákra képzés: $\mathbf{x_{pixel} = x_{norm} \cdot SCR_WIDTH}$, $\mathbf{y_{pixel} = y_{norm} \cdot SCR_HEIGHT}$

Függvényterv: *to_screen_pixels(p_intersect, p_origin, u_axis, v_axis, dimensions, resolution*
Felbontja a paramétereket: **Lw, Lh = dimensions**; **SW, SH = resolution**
Kiszámítja **w = p_intersect - p_origin** vektort
Skaláris szorzással meghatározza a lokális koordinátákat: **x_2d = w * u_axis; y_2d = w * v_axis**
Kiszámolja és visszadja: **x_px = np.clip(x_2d / Lw) * SW, y_px = np.clip(y_2d / Lh) * SH**
Eredménye egy 2 dimenziójú vektor **p_pixel** amely a metszett pont pixelkoordinátája

Folyamat röviden

$P_{INTERSECT}$ → [eltolási vektor 3D origóhoz képest] → [vetítés a síkra] → [normalizálás és korlátozás] → [pixel koordinátákra képzés] → P_{pixel}

A kapott (x,y) koordinátát Kalman filterrel stabilizáljuk.

Cél: A képernyőre vetített mutató mozgásának stabilizálása a mérési zajok és az apró, véletlenszerű koordináta-ingadozások csökkentésével. A szűrő a korábbi pozíció és a pillanatnyi mérés alapján becsüli meg a mutató aktuális helyzetét.
Bemenet: A mutató mért képernyőkoordinátái a k. képkockában: $z_k = (x_k, y_k)$.
Kimenet: A szűrt képernyőkoordináták: (x_{out}, y_{out}).
A szűrő állapotvektora a mutató pozícióját és sebességét tartalmazza:
$x_k = [p_{x,k}, p_{y,k}, v_{x,k}, v_{y,k}]^T$
Ahol $p_{x,k}$ és $p_{y,k}$ a becsült pozíciót, $v_{x,k}$ és $v_{y,k}$ pedig a becsült sebességet jelölik.
1. Előrejelzés
A szűrő az előző képkocka korrigált állapotából megbecsüli az aktuális állapotot:
$x_k^- = \Phi_k \cdot x_{k-1}$
A Φ_k állapotátmeneti mátrix az előző állapot alapján modellezi a pozíció és a sebesség változását. Az előrejelzés bizonytalanságát a következő összefüggéssel frissítjük:
$P_k^- = \Phi_k \cdot P_{k-1} \cdot \Phi_k^T + Q_k$
A P_k a becslés bizonytalanságát, a Q_k pedig a folyamat zaját modellezi.
2. Korrekció
Az aktuális mérés és az előrejelzés különbsége alapján korrigáljuk a becsült állapotot. A mérési modell:
$z_k = H \cdot x_k + \mu_k$
A H a mérési mátrix, μ_k pedig a mérési zaj. Mivel a bemenet csak a pozíciót tartalmazza, a mérési mátrix az állapotvektor pozíciókomponenseit választja ki.
A Kalman-erősítés meghatározza, hogy a szűrő mennyire támaszkodjon a mérésre az előrejelzéshez képest:
$S_k = H \cdot P_k^- \cdot H^T + R_k$
$K_k = P_k^- \cdot H^T \cdot S_k^{-1}$
Az R_k a mérési zaj kovarianciamátrixa, a K_k pedig a Kalman-erősítés.
A becslés korrigálása:
$x_k = x_k^- + K_k \cdot (z_k - H \cdot x_k^-)$
A korrigált bizonytalanság:
$P_k = (I - K_k \cdot H) \cdot P_k^-$
3. Kimenet
A korrigált állapot pozíciókomponensei adják a megjelenítendő koordinátákat:
$x_{out} = p_{x,k}$
$y_{out} = p_{y,k}$

Függvényterv:
`init_kalman()`: az állapotvektor, a kovarianciamátrixok és a szűrő paramétereinek inicializálása.
`update_kalman(x, y)`: az új mért koordináták fogadása, az előrejelzés és a korrekció végrehajtása, majd a szűrt koordináták visszaadása.

Folyamat röviden

Mért koordináták → állapot előrejelzése → mérés feldolgozása → becslés korrigálása → szűrt koordináták továbbítása a Qt overlay számára.

Egy átlátszó, always-on-top Qt overlay kirajzolja a virtuális pointert bármilyen alkalmazás fölé.

A megoldást egy külön, keret nélküli Qt-ablakként érdemes felépíteni: ez a teljes virtuális asztalt lefedi, áttetsző marad, nem fogja meg az egérkattintásokat, és a `paintEvent()` rajzolja rá a virtuális pointert.
A megvalósítás három fő részből áll:
Overlay-ablak létrehozása: keret nélküli, áttetsző és mindig felül maradó Qt-ablak.
Pointer kirajzolása: a `QPainter` segítségével egy kör, célkereszt vagy egyedi kurzor rajzolása.
Pozíció frissítése: minden új becsült koordinátánál frissül a pointer helye, majd a Qt újrarajzolja az overlayt.